

A Simple Recipe for Writing Decent Recursive Descent Parsers

Luyu Cheng ✉ 

The Hong Kong University of Science and Technology (HKUST), China

Lionel Parreaux ✉ 

The Hong Kong University of Science and Technology (HKUST), China

Abstract

Parsing *well-designed* computer languages should not be a hard problem, be it for humans or for machines. This is not a new idea: in 1973, Vaughan R. Pratt argued against formalistic grammar specifications and in favor of a more intuitive and meaningful approach to designing and parsing syntax. In this Pearl, we take the reader on a journey through handwritten recursive descent parsing, revisiting Pratt's original philosophy in a modern, statically-typed functional programming language. Contrary to many existing tutorials on the subject, we do not stop at simple expression languages: we also discuss how to tackle the full syntax of a simple programming language while avoiding the pitfalls of ad-hoc implementations. Indeed, a downside of recursive descent parsing is that the specification of what the parser accepts is written in code, which may contain subtle bugs and is not easily accessible to end users. We describe a simple recipe for architecting extensible recursive descent parsers that can automatically produce a readable representation of the syntax specification. We illustrate our approach by implementing a parser for a variant of Caml Light. Overall, this paper serves both as a pedagogical introduction to Pratt parsing in a modern programming language and as a practical guide to programmers who just want to implement, without unnecessary headaches, a computer language that is easy to parse and easy to read.

2012 ACM Subject Classification Software and its engineering → Parsers; Software and its engineering → Syntax

Keywords and phrases Parsing, Pratt Parsing, Operator Precedence, Recursive Descent

Digital Object Identifier 10.4230/LIPIcs.ECOOP.2026.30

Category Pearl/Brave New Idea

Related Version *Full Version*: <http://lptk.github.io/recursive-descent-parsing-paper>

Supplementary Material *Software (Source Code)*: <https://doi.org/10.5281/zenodo.19783892>

InteractiveResource (Online demonstration):

<https://recursive-descent-parsing.mlscript.dev> [3]

Software (ECOOP 2026 Artifact Evaluation approved artifact):

<https://doi.org/10.4230/DARTS.12.1.17>

Acknowledgements We thank the anonymous reviewers for their careful reading and feedback.

1 Introduction

Parsing is commonly encountered through two entry points. One is the theory-first route: context-free grammars, pushdown automata, and families of algorithms such as $LL(k)$ and $LR(k)$ [11]. The other is the tool-first route: parser generators (e.g., Yacc, Bison, ANTLR) and parser-combinator libraries (e.g., Parsec). Of course, there exist excellent courses and textbooks that connect these perspectives. For many learners – especially those starting from short tutorials or documentation – the connection between the underlying ideas and a working implementation is often left implicit. As a result, neither route is always an ideal *first contact* for readers who want a concrete mental model of *how parsing works in practice*.



© Luyu Cheng and Lionel Parreaux;

licensed under Creative Commons License CC-BY 4.0

40th European Conference on Object-Oriented Programming (ECOOP 2026).

Editors: Robbert Krebbers and Alexandra Silva; Article No. 30; pp. 30:1–30:28

Leibniz International Proceedings in Informatics



LIPICs Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany



Tool-first entry points (parser generators and combinator libraries) are popular because they let users obtain working parsers quickly [21]. For beginners, this speed can come at a cost: the mechanism – states, conflicts, or backtracking behavior – often remains implicit until something fails, at which point diagnostics may be expressed in terms of internal machinery rather than user intent. In teaching, facility with a tool does not automatically translate into a robust mental model of parsing.

On the other hand, parsing theory is essential and should not be downplayed. However, many introductions emphasize the declarative viewpoint (grammars and properties) more than the engineering trade-offs that appear in real implementations: disambiguation conventions (such as precedence), pragmatic restrictions for performance, and occasional ad-hoc handling for corner cases. This paper is about making that implementation viewpoint explicit, without giving up structure.

Despite the advanced state of the art in formal language theory and automated parsing tools, handwritten recursive descent parsers are still the most widely used approach for real-world industrial programming languages¹ [26, 27]. But beyond industrial use, many people still write parsers from scratch for educational or even recreational purposes. Blog posts, side projects, and toy interpreters often center around writing a parser by hand, not because tools are not available, but because people love to see how a concrete stream of characters turns into a structured syntax tree. Therefore, a particularly effective way for learners to build a reliable mental model of parsing is to construct a small recursive descent parser from scratch at least once.

There is an overwhelming number of tutorials on how to write recursive descent parsers on the Web. However, a brief survey of existing parser tutorials (in the supplementary appendix) suggests that most stop at expression parsing, few cover statements, declarations, and blocks, and almost none treat patterns or types as first-class parsing problems. The survey is not meant to be exhaustive; it simply records which syntactic categories each tutorial makes explicit and whether the parser structure remains inspectable from the source code. The tutorials in textbooks are often written in low-level languages like C, or dynamically typed languages like Python or JavaScript. For example, Crockford [6] and Pratt [23] rely on dynamic object extensions, making it hard to adapt the code to statically typed languages. Moreover, many tutorials walk through expression parsing and abruptly stop when things get interesting, leaving beginners who want to build complete language parsers with no obvious way to proceed. They leave out an important middle ground.

Pratt’s original work [23] was mostly remembered for its top-down operator precedence parsing component, but arguably an equally important component of his paper was his proposed philosophy of parsing, which could be summarized as follows:

- *Syntax should serve semantics, rather than the other way around.* Pratt encouraged the use of “semantic tokens”: each token knows how it should behave and how to parse its “arguments” e.g., the following tokens and expressions. In this sense, precedence is not an ad-hoc hack but a semantic *disambiguation policy*.
- *Parsing should be modular, direct, and comprehensible.* The core parsing routine should be tiny, with most language-specific complexity factored into token- and rule-definitions. This modularity is about separating *syntax recognition* from later phases (typechecking, evaluation, compilation), not about assigning multiple meanings to the same syntax.

¹ As stated in ANTLR4’s own FAQ, “almost no one uses parser generators to build commercial compilers” (<https://github.com/antlr/antlr4/blob/master/doc/faq/general.md>).

Building on the principles above, we present a tutorial recipe for writing flexible recursive-descent parsers in a modern statically typed language, using Pratt parsing as our running example. In other words, we are not proposing a replacement for declarative language definitions; rather, we show a disciplined way to *realize* such definitions as a small, extensible recursive-descent implementation. We first focus on parsing operators with precedence in Section 2, which can be thought of as a basic *Pratt parsing* tutorial, and which proposes a novel way of defining the precedence and associativity of multi-character operators. In Section 3, we investigate how to extend this simple approach to a larger general-purpose programming language in a principled *rule-based* way. It combines four advantages:

- Making the parser behave predictably, avoiding the boilerplate of plain recursive descent and the corresponding bugs that may arise from it.
- Allowing the parser to be extended with new rules, which can add syntax features; these rules can even be created dynamically by the parsed program.
- Retaining advantages of traditional recursive descent parsers, such as the ability to yield precise and informative error messages, the ease of debugging, and the possibility of adding custom ad-hoc parsing logic where necessary.
- Enabling the automatic generation of simple syntax specifications in the form of a set of *railroad diagrams* (also called syntax diagrams), a schematic visualization of grammars [5], along with a precedence table. These are easily consumed by users of the language and are enough for them to fully understand the syntax recognized by the parser.

Finally, we briefly review some related work in Section 5.

In our experience, the approach is practical for *prototyping and teaching*: it yields a readable, modular recursive-descent parser while remaining extensible. We aim for this paper to serve as a useful reference for learners who want to understand how parsing works in practice and to build a complete parser for their own languages from scratch. We also hope that language designers can find this paper useful when they want to implement extensible syntax in a minimal and comprehensible way.

Implementation

To illustrate our approach, we have implemented a parser for a simple language based on Caml Light [16, 15], a lightweight and portable implementation of the core Caml language developed in the early 1990s as a precursor to OCaml. Caml Light is a small, fully bootstrapped bytecode system with a C runtime and interpreter, which historically made it easy to port and practical as a self-contained reference point for studying real-world language syntax. We adapt its syntax slightly to fit our philosophy². Specifically, we make the language more uniform by parsing patterns and terms using the same rules (though it would be perfectly possible to use separate rules) and we make the language extensible through directives. Our implementation covers the full Caml Light language, suggesting that the technique scales beyond toy expression grammars.

An interactive web demonstration of our parser is available as supplementary material. The user can type in code in the left pane and see the resulting syntax tree in the right pane. The complete set of automatically generated railroad diagrams (partially depicted in Figure 3) can be found in the Diagrams tab of the online demo.

² We adapt Caml Light slightly to better fit our presentation. One could call this language *PrattyCaml*!

tokens	A	+		B		*		C
binding powers		3	4	$\nrightarrow$	$\rightarrow$	5	6	

(a) Precedence: $*$ binds tighter than $+$.

tokens	A	+		B		+		C
binding powers		3	4	$\leftarrow$	$\nrightarrow$	3	4	

(b) Associativity: $+$ groups to the left.

■ **Figure 1** Binding powers in action: precedence and left-associativity in two example parses.

2 Simple Top-Down Operator Precedence Parsing

We first describe a simple variation on the original top-down operator precedence parsing described by Pratt [23], now also informally known as *Pratt parsing*, using a high-level programming style to make our description easy to understand. We use MLscript and explain its relevant features as we go along.

To keep the presentation focused, this section uses an expression-only language (literals, variables, parentheses, and extensible infix operators); Section 3 lifts the same idea to the syntax of whole languages.

2.1 Binding Power

A simple and general way of reasoning about precedence and associativity is to rephrase these two concepts into the unified notions of *left* and *right binding power*. An operator may bind more strongly on its left or right side, which determines its associativity, and the difference in binding power between different operators determines the classical notion of precedence. For instance, we might say that the binding powers of $+$ are (3, 4) and those of $*$ are (5, 6), where the first component of the pair is the left binding power and the second component is the right binding power.

We visualize how this works through two examples. First, Figure 1a is the table representing the parsing of expression $A + B * C$. The main question to answer here is whether B goes with $+$ on its left or with $*$ on its right. Since the right binding power of $+$ is 4 and the left binding power of $*$ is 5, the answer is the latter, which explains the interpretation of that expression as $A + (B * C)$.

Next, consider the parsing of expression $A + B + C$ shown in Figure 1b. By the same reasoning, this time B goes with the first $+$ since the right binding power of the first $+$ sets the current threshold to 4, while the left binding power of the next $+$ is only 3. Because 3 is not greater than 4, the continuation stops and B is returned leftward.

In Pratt's original implementation in top-down operator precedence, as well as many later implementations, each operator has only one binding power, and the associativity of an operator is determined by how the loop (or recursion) compares the current binding power and the binding power of the next operator: $<$ tends to make same-precedence operators group left, and $\leq$ tends to make them group right. Our approach makes associativity *data-driven* and more flexible; Section 2.3 shows another benefit.

2.2 A Simple Implementation

We exemplify using a mathematical expression calculator that supports variables. Throughout the rest of this paper, we interleave explanations with code snippets with colored indentation guides to help the reader follow the code structure.

Tokenization

It is a common practice to define parsers operating on a stream of *tokens*, rather than directly on streams of characters. In MLscript, which synthesizes functional and object-oriented programming, we represent an algebraic data type (a sum-of-product type) by defining a `data class` and listing its `constructor`s.

```
data class LiteralKind with
  constructor { Integer, Float, Character, String }

data class BracketKind with
  constructor { Angle, Round, Square, Curly, BeginEnd }

data class Token with constructor
  Literal(kind: LiteralKind, literal: Str)
  Identifier(name: Str)
  Symbol(value: Str)
  Open(kind: BracketKind)
  Close(kind: BracketKind)
```

For the Caml Light parser in this paper, `LiteralKind` covers integers, floating-point numbers, characters, and strings. Our tokens are simple and relatively language-agnostic. They intentionally do not include a hard-coded list of keywords and operators: *symbolic* identifiers like `->` are represented as `Token.Symbol(">")` and *non-symbolic* identifiers like `foo` are represented as `Token.Identifier("foo")`. We distinguish between identifiers and symbols because they lead to two different parsing strategies in Section 3. Lastly, `Token.Open` and `Token.Close` represent opening and closing brackets of various kinds. Parentheses are treated as atomic tokens, as they are not associated with any user-definable parsing logic. We only present token definitions in this section; the lexer implementation is not essential here and appears in the supplementary appendix

Abstract Syntax Tree

We employ the following abstract syntax tree to represent expressions. Among them, `Tree.Error` is used to mark errors in the syntax tree; while `Tree.Empty` indicates that a parser routine did not consume any tokens and did not produce any syntax tree either.

```
data class Tree with constructor
  Literal(kind: LiteralKind, literal: Str)
  Reference(name: Str)
  Application(callee: Tree, arguments: Stack[Tree])
  Error(tree: Option[Tree], message: Str)
  Empty
```

In MLscript, `Stack` is the functional list, which can be constructed inductively using the `Nil` and `Cons` constructors, or more conveniently using the `::` operator. For example, expression `a + b * c` is parsed as³ `Application(Reference("+"), Reference("a") :: Application(Reference("*"), Reference("b") :: Reference("c") :: Nil) :: Nil)`.

³ This illustrative AST omits the `Tree.` qualifiers from `Application` and `Reference`.

Operator table

In the original paper of top-down operator precedence parsing, the entire syntax and semantics of a language is expressed by attaching two procedures (*nud* and *led*⁴) to each token. Associating tokens directly with semantic procedures is a very general approach – one can either evaluate the expressions on the fly or directly translate them into other target languages – but is also too flexible. In the architecture of compilers and interpreters, people usually implement parsing as a separate stage, rather than interleave evaluation or translation within parsing.

Since our parser returns an abstract syntax tree, the table only needs to associate operator tokens and their binding powers. We first define a class to represent operator information.

```
data class Operator(name: Str, lbp: Int, rbp: Int, unary: Bool)
```

Then, we store operators in a mutable map keyed by operator name.

```
let operatorTable = Map.fromIterable of array of
  ["+", Operator("+", 3, 4, true)]
  ["-", Operator("-", 3, 4, true)]
  ["*", Operator("*", 5, 6, false)]
  ["/", Operator("/", 5, 6, false)]
  ["**", Operator("**", 8, 7, false)]
```

`Map.fromIterable` creates an immutable map from the given iterable collection of key-value pairs and `array` creates an array from the given elements. We then define a function to retrieve operators from the table. Here `|>` is the pipe operator (apply the left-hand value to the function on the right). One can retrieve an operator from the table using `op of "**"`⁵.

```
fun op(name: Str): Option[Operator] = operatorTable |> Map.get(name)
```

Lastly, we define the binding powers of prefix operators.

```
let prefixPower = 9
```

To parse prefix operators correctly, the prefix binding power should be higher than most infix operators, so it grabs its operand tightly. For example, `-`, as a prefix operator, should bind tighter than `*`, so `-a * b` parses as `(-a) * b`. For now, `prefixPower` is manually set. We will see how to extend this architecture to support user-defined operators and automatically assign binding powers in Section 3.

Parsing functions

Now we begin the main parsing function.

```
fun parse(tokens: Stack[Token]): Tree =
  // Continued below
```

For brevity, we store the remaining tokens in a mutable variable.

⁴ They are abbreviations for *null denotation* and *left denotation*, and respectively handle the cases when the token is used at the prefix and infix positions.

⁵ The `of` operator is used to apply a function to arguments without parentheses. Its binding power is very low, and it is right-associative, making it similar to `$` in Haskell but more readable.

```
let mut currentTokens = tokens
```

We also provide a `peek` function to look at the next token,

```
fun peek: Option[Token] = Stack.head(currentTokens)
```

and a `consume` function to consume the next token. Here `_` is a wildcard (ignore a matched value), and `::` deconstructs a non-empty list into its head and tail.

```
fun consume: Unit = if currentTokens is _ :: rest then set currentTokens = rest
```

The parsing functions are now split into only two mutually recursive functions, `expr` and `exprCont` that keep track of the binding power `bp` – this is the binding power that an operator needs to be considered part of the currently parsed subexpression.

The helper `peek` returns an `Option[Token]`⁶: `Some` wraps a present token and `None` denotes end-of-input. The following code pattern-matches on `peek` and the first two cases of `expr` handle literals and identifiers.

```
fun expr(bp: Int): Tree = if peek is
  Some of
    Token.Literal(kind, literal) then
      consume
      exprCont(Tree.Literal(kind, literal), bp)
    Token.Identifier(name) then
      consume
      exprCont(Tree.Reference(name), bp)
```

Then, if the next token is a symbol that is marked as unary in the operator table, we call `expr` with the binding power of prefix operators to parse the operand, wrap the result as a prefix-operator application, and then call `exprCont` to resume parsing continuations.

```
Token.Symbol(name) and op of name is Some(Operator(_, _, _, true)) then
  consume
  exprCont(Tree.Application(Tree.Reference(name), expr(prefixPower) :: Nil), bp)
```

Next, if the next token is an opening parenthesis, we consume it, parse the inner expression with `bp 0`, expect a closing parenthesis, and then resume parsing continuations outside the parentheses.

```
Token.Open(Round) then
  consume
  let tree = expr(0)
  if peek is Some(Token.Close(Round)) then
    consume
    exprCont(tree, bp)
  else Tree.Error of Some(tree), "Missing closing parenthesis"
```

Finally, if the next token is neither a literal, an identifier, an operator, nor an opening parenthesis, or if there are no more tokens, we report an error.

```
token then Tree.Error of None, "Unexpected token " + render(token)
None then Tree.Error of None, "Unexpected end of input"
```

⁶ Type `Option[T]` is comparable to `Maybe a` in Haskell, `'a option` in OCaml, or `Option[T]` in Scala.

In turn, `exprCont` looks at the next token and checks whether it names an operator in the table. If so, it retrieves the operator's binding powers and compares its left binding power against the current `bp`. When `lbp > bp`, `exprCont` consumes the operator and parses the right-hand side by calling `expr` with `rbp` as the new `bp`. It then builds a `Tree.Application` node and calls itself recursively to parse any following operators. In pattern `Some(:lbp, :rbp)`, `:` is record-field binding: it matches fields `lbp` and `rbp` and binds them to same-named variables (i.e., `Some({ lbp: lbp, rbp: rbp })`).

```
fun exprCont(acc: Tree, bp: Int): Tree =
  if peek is Some(Token.Symbol(name)) and
    op of name is Some(:lbp, :rbp) and lbp > bp then
    consume
    exprCont(Tree.Application(Tree.Reference(name), acc :: expr(rbp) :: Nil), bp)
```

Otherwise – if the next token is not an operator, or if `lbp <= bp` – `exprCont` returns the accumulated expression `acc` unchanged.

```
else acc
```

The last few lines of code bootstrap parsing and reject any remaining tokens.

```
let result = expr(0)
if peek is
  Some(token) then Tree.Error of Some(result), "Unexpected token " + render(token)
  None then result
```

Lastly, we provide a step-by-step example of how the parsing functions work.

► **Example 1.** We illustrate how the algorithm works by comparing the step-by-step parsing of two expressions: `A + B * C` and `A + B + C`. The key difference is how `B` binds: with `*` (higher precedence), `B` binds to the right; with `+` (same precedence, left-associative), `B` binds to the left. As is shown in Figure 2, the critical moment occurs when parsing `B`: in the left column, the next token `*` has left binding power 5, which is greater than the current binding power 4, so parsing continues rightward to capture `C`. In the right column, the next token `+` has left binding power 3, which is *not* greater than 4, so `B` is returned and combined with `A` before proceeding.

2.3 User-Defined Operators

A nice property of Pratt parsing with binding power is that it can easily handle *user-defined operators*. Some programming languages allow users to define their own operators. There are two main different design philosophies when it comes to specifying their precedence and associativity:

1. The language mandates uniform rules of precedence and associativity. Examples of this school of thought include Scala [25] and OCaml [20]. This has the advantage of being predictable. The parser is also easier to implement.
2. The language lets users specify custom precedence and associativity values for their operators. Examples of this include Haskell, Rocq, and Lean. This is very flexible, but unfamiliar operators can make code harder to read.

We adopt the former approach, as this article is a simple tutorial focused solely on parsing. Languages like OCaml and Scala determine the precedence of an operator based on its *first character*. Each language has a corresponding precedence table; for example, operators starting with `|` bind less tightly than those starting with `&`.

A + B * C				A + B + C			
tokens	function call	bp	acc	tokens	function call	bp	acc
Both expressions parse identically until <code>exprCont("B", 4)</code>							
A,+,B,*,C	<code>expr</code>	0		A,+,B,*,C	<code>expr</code>	0	
+,B,*,C	<code>consume</code>			+,B,*,C	<code>consume</code>		
+,B,*,C	<code>exprCont</code>	0	"A"	+,B,*,C	<code>exprCont</code>	0	"A"
B,*,C	<code>consume</code>			B,*,C	<code>consume</code>		
B,*,C	<code>expr</code>	4		B,*,C	<code>expr</code>	4	
*,C	<code>consume</code>			*,C	<code>consume</code>		
*,C	<code>exprCont</code>	4	"B"	*,C	<code>exprCont</code>	4	"B"
Paths diverge: the left binding power of * 5 > 4, but the left binding power of + is 3 < 4							
C	<code>consume</code>			+,C	<code>return</code>	4	"B"
C	<code>expr</code>	6		+,C	<code>exprCont</code>	0	(+ "A" "B")
	<code>consume</code>			C	<code>consume</code>		(+ "A" "B")
	<code>exprCont</code>	6	"C"	C	<code>expr</code>	4	
	<code>return</code>	6	"C"		<code>consume</code>	4	
	<code>exprCont</code>	6	(* "B" "C")		<code>exprCont</code>	4	"C"
	<code>return</code>	6	(* "B" "C")		<code>return</code>	4	"C"
	<code>exprCont</code>	0	(+ "A" (* "B" "C"))		<code>exprCont</code>	0	(+ (+ "A" "B") "C")
	<code>return</code>	0	(+ "A" (* "B" "C"))		<code>return</code>	0	(+ (+ "A" "B") "C")

■ **Figure 2** Stepwise parsing process of `A + B * C` and `A + B + C` side-by-side. The leftmost column shows the remaining tokens to parse, the next column shows the function being called, the next column shows the current binding power, and the rightmost column shows the accumulated expression so far. We use S-expression notation to represent the parse trees being built for the sake of conciseness.

Ambiguous cases

Scala operators are *right-associative* when they *end with a colon* `:`, whereas the associativity of OCaml operators is also determined by the first character of the operator. A question arises when mixing operators of the same precedence but different associativity. For instance, in Scala, `::` (list cons) is right-associative and has the same precedence as `:+` (list snoc), which is left-associative. So should `a :: b :+: c` parse as `a :: (b :+: c)` or as `(a :: b) :+: c`? Scala resolves this error by simply reporting an error in such cases.

```
scala> a :: b :+: c
-- [E041] Syntax Error: -----
1 | a :: b :+: c
  |      ^
  |:: (which is right-associative) and :+: (which is left-associative) have same
  ↳ precedence and may not be mixed
```

A nicer approach

A minor contribution of this paper is to propose a simple default policy, aligned with Pratt's idea of *binding power*. It is not a universal rule for operator design, but a convenient heuristic that works particularly well for *symmetric* operators and can be overridden when needed.

*The **first** and **last** characters of an operator should respectively determine the **left** and **right** binding powers of the operator.*

As an example, assume that character `|` has lower binding power than character `>`, which in turn has lower power than `+`. This means that the “forward pipe” operator `|>` binds more strongly on the right than on the left. So expression `a + b |> f |> g` should parse as `((a + b) |> f) |> g`, which is as intended for the so-called pipeline operator. On the other hand, one would expect the “reverse pipe” operator `<|` to work just as well, parsing `f <| g <| a + b` as `f <| (g <| (a + b))`. This is not the case in languages like Scala and OCaml, but our approach makes *symmetric operators* like `|>` and `<|` behave *symmetrically*.

30:10 A Simple Recipe for Writing Decent Recursive Descent Parsers

Coincidentally, this approach has the side effect of making operators like `->` behave as intended: `S -> T -> U` parses as `S -> (T -> U)` because, in standard mathematical notation, arithmetic operators such as `-` bind more tightly than comparison operators such as `>`.

The heuristic is most compelling for operators whose spelling is *structural* and symmetric, e.g. piping, composition, sequencing, and other “diagrammatic” operators. It is less appropriate for operators whose spelling follows pronunciation or convention (e.g. `<=` read as “less-than or equal to”), or for operators that are intentionally *asymmetric* in shape.

Character binding power table

Based on this idea, let us define a map of character power, where the `zipppingWithIndex`, `mapping`, and `flattening` functions of MLscript’s standard library are used to transform iterable collections of elements lazily (in a streaming fashion). Each string argument represents a group of characters, separated by spaces, that share the same binding power; the leading `...` in `...charGroups` indicates that `makeCharPowerMap` is variadic over character groups.

```
fun makeCharPowerMap(...charGroups: Array[Str]): Map[Str, Int] =  
  Map.fromIterable of charGroups  
    |> zipppingWithIndex  
    |> mapping of case [chars, bp] then chars.split(" ") |> mapping(char => [char, bp])  
    |> flattening
```

Now, we define a map of characters to binding powers. Note that we allocate the binding power of ML-style function application `f x` by the presentational symbol `"<"`.

```
let binaryOps = makeCharPowerMap of  
  ", ", "@", ":", "|", "&", "=", "/ \\", "^", "!", "< >", "+ -", "*"
```

Next, we define the unary-operator set: these operators can appear in prefix position.

```
let unaryOps = Set.fromIterable of "!~+-"
```

Then, we mark operators ending with these characters as right-associative.

```
let rightAssocChars = Set.fromIterable of ":", "/"
```

We define a function that checks if the given character is a valid operator character.

```
fun isOperatorChar(c: Str): Bool = binaryOps |> Map.has(c) or unaryOps |> Set.has(c)
```

Finally, we can rewrite `op` function to use the character binding power table. This function reads the first and last characters of an operator and returns its left and right binding powers, as well as whether it can be used as a unary operator. If the given string is not a valid operator, it returns `None`.

```
fun op(name: Str): Option[Operator] = if name.length > 0  
  and name.forall(isOperatorChar)  
  and name.at(0) is first and name.at(-1) is last  
  and binaryOps |> Map.get(first) is Some(lbp)  
  and binaryOps |> Map.get(last) is Some(rbp') then  
    let rbp = if rightAssocChars |> Set.has(last) then (rbp' - 1) else rbp'  
    let unary = unaryOps |> Set.has(name)  
    Some(Operator(name, lbp, rbp, unary))  
  else None
```

Above, the final line wraps the derived metadata in the same `Operator` class used by the table-based version of `op`.

With this extension, the parser of the mathematical expression calculator now supports a variety of operators with a sensible default precedence and associativity rules.

2.4 Summary

In this section, we built the minimal Pratt core: a small tokenizer, a binding-power table, and two mutually recursive functions – `expr` that parses an atom (a literal, variable, or parenthesized expression) and `exprCont` that repeatedly parses a continuation (an infix operator followed by its right-hand side).

This setup is great for “expression-shaped” syntax, but real languages quickly go beyond unary/binary operators: they have keyword-delimited constructs (e.g., `if-then-else`), multiple syntactic categories (terms, types, patterns, etc.), and reusable fragments whose syntax should be specified directly rather than inferred from parser implementations.

In the next section, we keep the same Pratt idea – parse an atom, then extend it by parsing continuations under a binding-power threshold – but we generalize the operator table into a first-class representation of grammar-like rules. This refactoring will rephrase `expr`/`exprCont` into a slightly different core, because the parser must now consult reified rules (not just an operator map) while it runs.

3 Going Beyond Expression Parsing

When using recursive descent parsing, its hard-coded parsing logic effectively becomes the specification of the syntax. For example, to parse `if-then-else` expressions, one typically writes a dedicated function `parseIfThenElse` that recognizes the `if` keyword, parses the condition, expects `then`, parses the consequent, and optionally consumes an `else` branch. Such code often mixes the intended syntax with ad-hoc special cases and repetitive boilerplate, forcing readers to study the implementation to recover the language definition. This approach is also error-prone: parsers may accept unreasonable inputs or reject valid ones.

Pratt encouraged the use of “semantic tokens”, where each token indicates how it parses its surrounding phrase. For example, token `if` is associated with a parsing action that parses the condition expression, then expects `then`, and finally parses the consequent and the optional `else` branch. The parser is created by assigning parsing functions to entries in the prefix and infix operator tables in a specific order. This makes the syntax modular and dynamically extensible. However, it can also make the syntax harder to *see* and harder for people to understand, because parsing functions are the only definition.

Can we keep Pratt’s modular, token-driven extensibility, while making syntax inspectable and teachable through first-class parsing rules?

3.1 Generalizing the Operator Table

Unary and binary expressions are just one aspect of parsing general-purpose programming languages. Many expressions are made of multiple keywords and involve more than two operands, for example, if-then-else expressions. There are also more kinds of syntax constructs, for example, types, statements, patterns, etc.

Recall the shape of the Pratt parser from Section 2: we first parse a smallest expression “atom”, and then we repeatedly extend it by consuming a continuation as long as the next token has sufficiently high binding power. There, continuations were infix operators from a table. Here, we keep the same control flow, but we generalize “what counts as a continuation” to include keyword-led phrases and references to other syntactic categories.

30:12 A Simple Recipe for Writing Decent Recursive Descent Parsers

Our approach is to generalize the operator table used in top-down operator precedence parsing into a reified representation of phrasal grammar rules. In this way, we still adhere to Pratt’s philosophy: the parsing rules remain modular and independent of the core logic, and their execution is driven by tokens.

This representation brings two additional benefits: it can generate both a complete precedence table for operators and a syntax diagram that clearly illustrates the language’s syntax, as we will see in the rest of this section.

We present the generalized representation first, and then show how the same Pratt-shaped loop reappears on top of it. Concretely, the two functions from Section 2 get lifted as follows.

- The atom-parsing function `expr(bp)` is split into two entry points: `parseRule(bp, rule)` for a known rule, and `parseKind(kind, bp)` for a named syntax kind. In both cases, `bp` is still the threshold.
- The role of `exprCont(acc, bp)` (“extend `acc` by parsing continuations”) becomes a loop that repeatedly applies a continuation function produced by the rule.

3.2 Representation of Parsing Rules

We add a new type `Rule` to represent a parsing rule, which is an array of *choices*. `Rule` is parameterized by the type of the parsing result (i.e., `A`).

```
data class Rule[A](choices: Array[Choice[A]])
```

Each choice is represented by the `Choice` class. Choices have the following constructors.

```
data class Choice[A] with constructor
```

- `End(value)` represents an end to the parsing rule, when there might be nothing more to parse, in which case the parser should return the `value`.

```
End(value: A)
```

- `Keyword(kw, rest)` represents a parsing rule that starts with keyword `kw` and then continues parsing according to `rest`.

```
Keyword(kw: Keyword, rest: Rule[A])
```

- `Ref(kind, f, bp, rest)` is the most powerful one: it parses according to the syntax kind referenced by `kind` (these will be, e.g., `"term"`, `"type"`, or user-defined kinds). The parsed result is processed by function `f` to produce the final result of type `A`.

```
Ref[B](kind: Str, f: (Tree, B) -> A, bp: Option[Int], rest: Rule[B])
```

The choice can only be chosen if `bp` is `None` or if the current parsing binding power is less than the value contained in `bp`.

Throughout the rest of this section, we use a list of helper functions (`rule`, `reference`, `term`, `end`, `keyword`) as shorthands of the `Choice` constructors. Most are thin wrappers, and can be invoked concisely using MLscript’s calling convention of records:

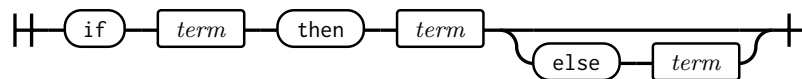
```
fun rule[A](...choices: Array[Choice[A]]) = Rule(choices)
fun end[A](value: A) = End(value)
fun keyword[A](kw: Keyword)(...choices) = Keyword(kw, rule(...choices))
fun term(fields)(...choices) = Ref("term", fields.process, fields.bp, rule(...choices))
```

The full helper set, including `reference`, appears in the supplementary appendix. The helper `term` specializes references to terms; `reference` follows the same pattern for other syntax kinds, and `end` supplies the result for a completed rule.

► **Example 2.** One can now define parsing rules like the one below for if-then-else expressions:

```
let ifThenElse = keyword(_if) of term(name: "if-then-else condition") of
  keyword(_then) of term(name: "if-then-else consequent") of // two alternatives:
    end // we can end here without 'else' or continue with 'else':
  keyword(_else) of term(name: "if-then-else alternative") of end
```

The rule above generates the following syntax diagram.



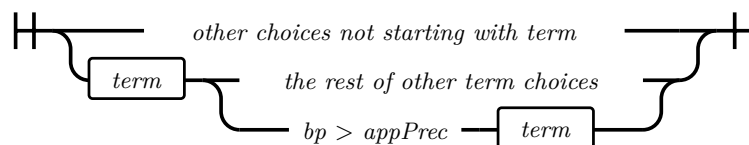
► **Example 3.** An example that top-down operator precedence cannot achieve is parentheses-less function application just like in ML family of languages. Recall that we have reserved a slot for function application when making the binary operator binding power table. We can first retrieve the reserved binding power for application.

```
let fnAppBp = binaryOps |> Map.get("<") |> Option.get
```

Then, we add an expression-rule choice using the reserved power as its outer threshold.

```
let exprRule = array of // ... other choices for expressions are omitted
  term of
    // ... other choices for expressions are omitted
  term of name: "application argument", bp: fnAppBp
```

The rule above generates the following syntax diagram. After we have parsed the first term, the second term can be parsed only when no other choices are applicable and the binding power for application is greater than the current binding power.



From the examples above, we can see that the representation of parsing rules is quite concise and readable. Another advantage is that we can automatically generate syntax diagrams from the rules. The purpose of diagrams is twofold: language designers can verify rules by checking diagrams, and users can learn the syntax from them.

3.3 The Parser Framework

Before we dive into the details of how the parser handles the parsing rules, we first introduce the parser's framework, including needed utility methods, global registry of keywords and syntax kinds, and the bootstrapping process.

Keywords

We define a `Keyword` class to represent keywords. Each `Keyword` instance carries the left and right binding powers. Note that `lbp` and `rbp` are `Option[Int]`.

- If `lbp` is `None`, then the keyword cannot be used as the leading keyword of an expression, but can still be used as a delimiter in “multi-argument constructs”. For example, `else` keyword in `if-then-else` expressions can have `lbp` as `None`, but `if` needs a defined `lbp` to begin an expression.
- If `rbp` is `None`, then the following expression after the keyword takes the binding power we use before parsing the keyword.

```
data class Keyword(name: Str, lbp: Option[Int], rbp: Option[Int]) with
  Keyword.all |> MutMap.insert(name, this)
```

We maintain a global registry of keywords in the companion module⁷ of `Keyword`.

```
module Keyword with
  val all: MutMap[Str, Keyword] = MutMap.empty
```

We also maintain a counter `bp`⁸ of binding powers, function `curr` to get the current binding power wrapped in `Some`, and function `next` to increment the binding power counter and return the new binding power wrapped in `Some`.

```
let bp = 0
fun curr = Some(bp)
fun next = set bp += 1; Some(bp)
```

We might reuse some binding powers if they are used for multiple keywords.

```
let base = curr // the lowest binding power
let semi = next
```

Thus, new `Keyword` instances automatically register themselves in the map `Keyword.all`.

```
val _terminator = keyword(";", base, base)
val _semicolon = keyword(";", semi, base)
val _comma = keyword(",", next, semi)
val appPrec = next // reserved for function application
val leftArrow = keyword("<=", next, base)
```

```
val rightArrow = keyword(">=", next, base)
val _colon = keyword(":", next, base)
// ... more keywords are omitted for brevity
```

Lastly, we define `max` to be greater than any assigned binding powers.

```
val max = set bp += 1; bp
```

The map of keywords `Keyword.all` can be used to generate a precedence table, which serves as a reference for users to understand how keywords and operators interact with each other. Readers can inspect this generated precedence table in the online demo.

⁷ A module with the same name defined alongside the `Keyword` type, used for shared definitions.

⁸ The difference between `let` and `val` is that the former is private to the class instance.

Rules

A rule contains multiple choices.

```
data class Rule[A](choices: Array[Choice[A]]) with
```

The following methods help obtain the appropriate choices from a rule. For example, seeing a keyword `k` asks `keywordChoices` `|> Map.get(k)` for the continuation rule.

```
val keywordChoices = Map.fromIterable of choices.flatMap of
case Keyword(keyword, rest) then [[keyword.name, rest]] else []
```

Method `endChoice` returns the first `End` among `choices`, if any, and extracts its `value`.

```
val endChoice: Option[A] = choices.firstDefined of
case End(value) then Some(value) else None
```

Method `refChoice` returns the first reference choice in the rule.

```
val refChoice: Option[Ref[A]] = choices.firstDefined of
case Ref as ref then Some(ref) else None
```

Method `afterRef` returns the rule that we should continue with after we have successfully parsed its `refChoice`. This serves the same purpose as `exprCont` shown in Section 2. We will see its usage in parsing rules with left recursion in Section 3.3.

```
val afterRef: Option[[Str, Rule[Option[Tree -> A]]]] = if refChoice is
Some(Ref(kind, f, None, rest)) then Some of
  tuple of kind, rest.map of case
    Tree.Empty | Tree.Error then None
    result then Some(tree => f(tree, result))
  None then None
```

The method `firstDefined` on arrays applies a function to each element of the array and returns the first element that is not `None`.

Syntax Kinds

Users can define their own syntax kinds by adding entries to `syntaxKinds`, a mutable map from syntax kind names to their `Rules`.

```
let syntaxKinds: MutMap[Str, Rule[Tree]] = MutMap.empty
```

Existing syntax kinds can be extended by adding more choices; 3.6 discusses this.

```
fun define(kind: Str)(...choices: Array[Choice[Tree]]) =
  syntaxKinds |> MutMap.updateWith(kind) of case
    None then Some(Rule(choices))
    Some(rule) then Some(rule.extend(choices))
```

The choices of a syntax kind can reference the same syntax kind, enabling recursive definitions that can express repetitive constructs.

30:16 A Simple Recipe for Writing Decent Recursive Descent Parsers

► **Example 4.** The following rule defines pattern matching arms in Caml Light.

```
define("simple-matching") of // The 'simple-matching' syntax kind ...
  reference(to: "pattern") of // starts with a pattern followed by ...
    name: "case body"
    keyword(rightArrow) of term(name: "case tail") of // '->' and a term, and ...
      end // either end here or continue with a '|' and a simple-matching.
      keyword(_bar) of reference(to: "simple-matching")
```

The syntax diagram of syntax kind `"simple-matching"` is shown below.



Function `parseKind` handles the built-in syntax kinds and retrieves the parsing rule for a syntax kind and delegates the parsing to it. The parser has a few built-in syntax kinds for terminal symbols, such as identifiers (`"ident"`) and literals (`"literal"`). They are atomic and thus cannot be extended dynamically, but we can extend them by adding hard-coded logic to this function.

```
fun parseKind(kind: Str, bp: Int): Tree = if
  kind is "ident" and peek is
    Some(Token.Identifier(name)) and Keyword.all |> MutMap.get(name) is None then
      consume
      Tree.Reference(name)
    Some(token) then Tree.Error of None, "expect an identifier but found " + token
    None then Tree.Error of None, "expect an identifier but found the end of input"
  kind is "literal" then // ... omitted for brevity
```

The last case handles user-defined syntax kinds. It first calls `parseRule` to parse the rule. Then, it checks if there is a continuation rule and the continued kind is the same as the current kind. If so, it keeps parsing the rest of the rule until the result is empty or erroneous.

```
syntaxKinds |> MutMap.get(kind) is Some(rule) then
  let mut acc = parseRule(bp, rule)
  if rule.afterRef is Some([kind', rest]) and kind' == kind then
    while parseRule(bp, rest) is Some(op) do set acc = op(acc)
  else acc
  else fatal("Unknown syntax kind: \" + kind + "\"")
```

This encodes the continuation that, in Section 2, was implemented directly by `exprCont`: after parsing the referenced subexpression, it describes how to continue parsing and how to combine the next piece with the accumulated result. This straightforward technique can even parse grammar rules with direct left-recursion (i.e., self-referential rules), as long as the `afterRef` does not overlap with the current `rule`.

Bootstrapping

To start the parser, we first parse the input using the "program" syntax kind (which contains choices for top-level constructs). After parsing, we check that all tokens have been consumed and report a syntax error if any tokens remain.

```
let program = parseKind("program", 0)
if peek is
  Some(token) then Tree.Error of Some(program), "expect end of input; found " + token
  None then program
```

3.4 Parsing Rules

In this section, we elaborate on how the parser handles the parsing rules. Besides `expr` and `exprCont`, we have another parsing routine `parseRule`, which accepts a `Rule` instance, parses according to the rule, and returns an abstract syntax tree.

```
fun parseRule(bp: Int, rule: Rule[Tree]) = if peek is
```

The first case handles keyword-based rules. It matches when three conditions are met: the current token is an identifier or symbol, the token spelling is defined as a keyword, and there is a choice in the rule that starts with this keyword. When these conditions are satisfied, we consume the keyword token and continue parsing according to the `rest` of the rule.

```
Some(Token.Identifier(name) | Token.Symbol(name)) and
keyword.all |> MutMap.get(name) is Some(keyword) and
rule.keywordChoices |> Map.get(name) is Some(rest) then
  consume
  parseRule(bp, rest)
```

When the current token is not a keyword, we check if there is a reference choice. A reference choice specifies an entry binding power (`ebp`). We can only proceed if the current binding power (`bp`) is less than `ebp`. If this condition is met, we first parse the referenced syntax kind `kind` at the current binding power. Note that the parsed result from `parseKind` must not be an error or empty tree. On success, we continue parsing the rest of the rule and apply the process function to combine the parsed results into the final tree using `Ref`'s `f`.

```
Some(other) and rule.refChoice is Some(Ref(kind, f, ebp', rest)) and
let ebp = ebp' |> Option.getOrElse(Keyword.max)
ebp > bp and parseKind(kind, bp) as acc is ¬(Tree.Error | Tree.Empty) and
  parseRule(bp, rest) as tree is
  Tree.Error then tree
  else f(acc, tree)
```

However, if the current token is neither a keyword nor can be handled by a reference choice, we check if there is an end choice in the rule. If so, we return the value of the end choice.

```
rule.endChoice is Some(value) then value
else
  consume
  Tree.Error of None, "unexpected token " + other
```

Lastly, if the tokens are exhausted, we can only consider the end choice. If the end choice does not exist, we report an error of unexpected end of input.

```
None and rule.endChoice is
  Some(value) then value
  None then Tree.Error of None, "unexpected end of input"
```

The function `parseRule` interprets our reified rules: it consumes keywords, follows at most one reference choice (when permitted by binding power), and otherwise falls back to an end choice. In the rest of the parser, `parseRule` is typically invoked indirectly via `parseKind`, but we also reuse it inside the dedicated expression parsers. The next subsection shows how the traditional Pratt-style routines for terms and types (`expr`/`exprCont` and `typeExpr`/`typeExprCont`) adapt to consult rules and keywords without changing shape.

3.5 Parsing Terms and Types

So far, we have described a generic mechanism for executing reified rules via `parseRule`, and for dispatching on syntax kinds via `parseKind`. However, Caml Light still benefits from keeping the dedicated Pratt-style entry points for terms and types: they provide the conventional “parse an atom, then greedily extend it” control flow, while delegating the parsing of individual constructs (keywords, applications, ascriptions, etc) to rules.

Accordingly, we retain the two pairs of routines: `expr` with `exprCont` for terms, and `typeExpr` with `typeExprCont` for types, but refactor them so that keyword-led constructs are parsed by calling `parseRule` on an explicit rule definition. Since the two pairs are analogous and the type fragment is simpler, we only show `expr` and `exprCont` below.

First, we define a rule for term expressions. It contains a set of choices, each of which can parse a specific kind of term expression, for example, if-then-else expressions, for-loops, and function applications we mentioned earlier.

```
val exprRule = rule of
  ifThenElse
  // Lambda expressions starting with 'fun'.
  keyword(_fun) of term of keyword(Keyword.rightArrow) of term
  // ... more choices starting with keywords are omitted for brevity
  term of

  // Tree.Application: <expr> <expr>
  term of
    process: (argument, _) => callee => Tree.Application(callee, argument :: Nil)
    name: "application argument"
    bp: Keyword.appPrec
  // Type ascription: <expr> : <type>
  keyword(Keyword._colon) of typeExpr of
    process: (rhs, _) => lhs =>
      Tree.Application(Tree.Reference(Keyword._colon.name), lhs :: rhs :: Nil)
    name: "right-hand side type"
  // ... more choices starting with terms are omitted for brevity
```

The rule has a choice that starts with terms, which enables parsing expressions that involve left-recursion, such as function applications, type ascription, and more kinds of expressions starting with terms.⁹ These “starts with terms” choices are precisely the ones consumed by the continuation loop in `exprCont`: after parsing an initial term, the parser repeatedly tries to apply such extensions until binding-power conditions fail.

Parsing expressions

Then we begin `expr`. The first case mirrors `parseRule`: when the next token is an identifier or symbol whose spelling is registered as a keyword, we consult the rule’s keyword choices.

```
fun expr(bp: Int): Tree = if peek is
  Some(Token.Identifier(name) | Token.Symbol(name)) and
  Keyword.all |> MutMap.get(name) is Some(keyword) and
  exprRule.keywordChoices |> Map.get(name) is
  Some(rule) and
  keyword.lbp |> Option.getOrElse(Keyword.base) > bp then
  consume
  exprCont(parseRule(keyword.rbp |> Option.getOrElse(bp), rule), bp)
else Tree.Empty
None then Tree.Empty
```

⁹ Note that `term of process: p, name: n, bp: p` in MLscript is syntax for a call with named arguments, i.e., syntax sugar for `term({process: p, name: n, bp: p})`.

Within this keyword case, if the keyword is not applicable at the current binding power or has no matching choice, we return `Tree.Empty` so that callers (notably `exprCont`) can treat it as “no extension possible” rather than a hard error. If the next token is instead a non-keyword identifier, we consume it as a variable reference.

```
Some(Token.Identifier(name)) then
  consume
  exprCont(Tree.Reference(name), bp)
```

The next case is for literal tokens. This case is not necessary in `typeExpr` and `typeExprCont`, as Caml Light types do not contain literals. Recall that the token constructor `Literal` carries a `kind` field to distinguish different kinds of literals.

```
Some(Token.Literal(kind, literal)) then
  consume
  exprCont(Tree.Literal(kind, literal), bp)
```

Lastly, the two error cases report unrecognized tokens and unexpected end of input.

```
Some(token) then Tree.Error of None, "unrecognized token: " + render(token)
None then Tree.Error of None, "unexpected end of input"
```

Parsing expression continuation

The implementation of `exprCont` follows the continuation loop from Section 2, but consults the rule representation (in particular, the reference choice) to decide which extensions are available. Before defining the loop, we first retrieve the reference choice from `exprRule`. This choice is the entry point for constructs that continue an already parsed left-hand side, such as applications, infix operators, and type ascriptions. Separating it from the prefix and atomic choices mirrors the `expr`/`exprCont` split in the basic Pratt parser.

```
let infix = exprRule.refChoice |> Option.flatMap of case
  Ref("term", process, None, rest) then { process: process, rule: rest }
  else fatal("Missing the infix rules")
fun exprCont(acc: Tree, bp: Int) = if peek is
```

The first case handles infix keyword rules.

```
Some(Token.Identifier(name) | Token.Symbol(name)) and
  Keyword.all |> MutMap.get(name) is Some(keyword) and
  infix.rule.keywordChoices |> Map.get(name) is Some(rule) and
  keyword.lbp |> Option.getOrElse(Keyword.base) > bp and rule.refChoice is
  Some(Ref(kind, process, ebp', rest)) and
  let ebp = ebp' |> Option.getOrElse(Keyword.max)
  ebp > bp then
    consume
    let rhs = parseKind(kind, keyword.rbp |> Option.getOrElse(bp))
    let restRes = parseRule(ebp, rest)
    exprCont(infix.process(acc, process(rhs, restRes)), bp)
  None then acc
```

The next case handles infix operators.

30:20 A Simple Recipe for Writing Decent Recursive Descent Parsers

```
Some(Token.Symbol(name)) and Keyword.all |> MutMap.get(name) is None and
  op of name is Some(:lbp, :rbp) and
    lbp > bp then
      consume
      let rhs = expr(rbp)
      exprCont(Tree.Application(Tree.Reference(name), acc :: rhs :: Nil), bp)
    else acc
```

Lastly, we have the case of expressions that starts with terms.

```
Some(token) and infix.rule.refChoice is
  Some(Ref(kind, process, ebp', rest)) and
    let ebp = ebp' |> Option.getOrElse(Keyword.max)
    ebp > bp and parseKind(kind, ebp) is
      Tree.Empty | Tree.Error then acc
      rhs then
        let restRes = parseRule(ebp, rest)
        exprCont(infix.process(acc, process(rhs, restRes)), bp)
      else acc
  None then acc
  None then acc
```

Toward fully Rule-driven Pratt parsing

Conceptually, `expr` and `exprCont` differ from `parseRule` only in one place: they add a dedicated branch for infix operators whose applicability is governed by binding powers (as opposed to keyword-led constructs, which are already handled uniformly via keyword choices). This suggests factoring operator precedence into the rule representation itself. Accordingly, we can extend `Choice` with a new constructor (added to Section 3.2):

```
Infix(op: Str -> Option[Operator], rhsKind: Str)
```

This constructor is already implemented in the artifact, where interested readers can inspect the corresponding source code. Here, `op` is the operator function queried by `exprCont`: given an operator name, it returns its `Operator` metadata when the name denotes an infix operator, and `None` otherwise. The operator's `lbp` and `rbp` fields determine binding power. The field `rhsKind` indicates which syntax kind should be parsed on the right-hand side when such an operator is recognized. With `Infix`, the remaining control flow of `expr`/`exprCont` can be expressed purely by rule dispatch: the “atom, then greedily extend” loop becomes just another reified continuation that consults `Infix` when it encounters a symbolic identifier. As a result, adding new precedence-driven syntax kinds no longer requires introducing new functions such as `expr`/`exprCont` (or `pattern`/`patternCont`); instead, one can declare new rules with `Infix` choices and reuse the same generic mechanism. This also naturally supports *cross-kind* infix expressions: for instance, the ascription operator `:` parses a term on the left, but continues with a type on the right, which is captured directly by setting `rhsKind` to `"type"` for that operator rather than hard-coding a special case.

3.6 Dynamic Extensibility

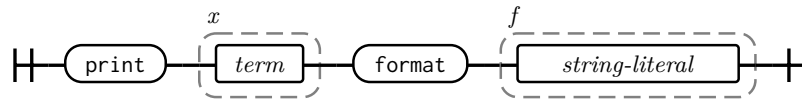
The parser we specified in this section is extensible in the sense that the language users (not the language designers) can extend the syntax on the fly.

We take Caml Light as an example, by making use of a handy construct called *directive* in its language specification. A directive is an open-ended command form that can “control the behavior of the compiler”. For instance, `#open "foo"` is a valid Caml Light directive. We

reuse this concept to support defining new keywords (through the `#keyword` directive) and extending existing syntax kinds. We add a new directive called `#extend`, which has three arguments: the target syntax kind, a sequence of keywords or syntax kind names, and the expression to be replaced at the use site.

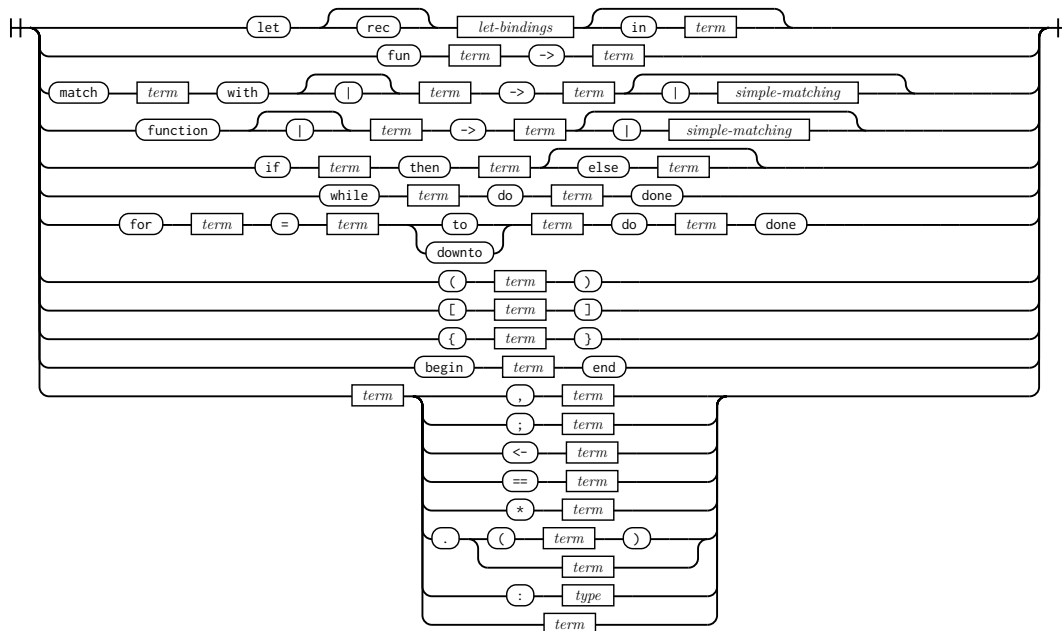
```
#keyword "print" None 10
#keyword "format" None None
#extend ["term",
  [keyword("print"), ["term", "x"], keyword("format"), ["string-literal", "f"]],
  [System.println (String.format f x)]]
```

The code above extends `term` by adding a new choice, starting with the keyword `print`, followed by a term (the argument to be printed), then the keyword `format`, and finally a string literal (the format string)¹⁰. The syntax diagram of the rule extended to `term` is shown below. The labeled parts `x` and `f` are the expressions to be replaced at the use site.



3.7 The Syntax Diagrams

As an example, we have written a small utility that can generate railroad diagrams for any given parsing rules constructed from our `Choice` and `Rule` data types. Figure 3 shows a compact, incomplete `term` syntax diagram for our Caml Light parser, generated with `railroad-diagrams` [1]. The complete generated diagram is available in the Diagrams tab of the online demo.



■ **Figure 3** A compact excerpt of the `term` parsing rules of our Caml variant.

¹⁰The syntax kind `"string-literal"` is built-in and recognizes a single string literal token.

3.8 Conclusion

This section generalizes Pratt parsing from Section 2 by moving from an operator table to a first-class representation of syntax. A *syntax kind* plays a role similar to a non-terminal in a grammar, but each kind is defined by an extensible `Rule` (a tree of `Choices`) rather than a fixed set of productions. The parser keeps the same Pratt-shaped control flow: `parseKind`/`parseRule` interpret these rules under a binding-power threshold, and continuations are driven by `afterRef`. Compared to top-down operator precedence, the result remains modular and open to extension, while making the language definition easier to inspect and reuse. As a practical benefit, the same rule representation can be rendered as syntax diagrams, providing a readable specification without re-reading parser code.

4 Historical Perspective and Discussions

In 1973, Vaughan R. Pratt [23] described a simple way of parsing operators with precedence in a *recursive descent* style, meaning that the parser is designed as a set of functions calling each other recursively. This technique – now often called *Pratt parsing* and closely related to *precedence climbing* – has since been rediscovered a few times, and remains a popular compiler implementation technique today. But Pratt’s original paper went further: it also argued for what he called *lexical semantics* instead of the traditional *syntactic semantics*. The former assigns meaning to individual marker tokens, while the latter assigns meaning to entire phrases as specified by a formal grammar in Backus–Naur form (BNF). BNF allows the same tokens to participate in different phrases, creating ambiguities as syntax forms interact in non-trivial ways. Pratt noted that, in practice, each language feature can often be associated with a specific unambiguous token (often a keyword), which is also how programmers tend to learn the syntax. This makes the surface language easier to read and explain.

4.1 A Syntax Design Philosophy and the Case of JavaScript

Many languages today follow Pratt’s vision to some extent, often without citing it explicitly, having moved away from the syntactic excesses of the C and C++ eras. Illustrious examples that mostly fit this philosophy are Algol, Pascal, and the ML-family languages. Sometimes, Pratt’s philosophy also lets us reinterpret the syntax of an existing language *a posteriori*: it may look problematic from the formal-grammar perspective, yet become unproblematic when viewed more “lexically”. Consider JavaScript, where function definitions start with the keyword `function`, as in:

```
function f(x) { return x + 1; }
```

but where function *expressions* can also start with the same keyword, as in:

```
let res = foo(function (x) { return x + 1; })
```

Notice that a function expression may omit its name, as seen above, but it may also specify a name, which is needed when the function is recursive.

If one tries to express this in a traditional BNF grammar, one encounters an apparent ambiguity: because expressions can be used as statements, a recursive descent parser seeing `function` in statement position cannot tell whether it is parsing a function definition or a function expression used as a statement. Resolving this naïvely suggests either arbitrary amounts of lookahead or a more complex parsing technique. In fact, *we need not resolve this*

ambiguity at all: the `function` keyword always introduces functions, which parse the same way. What differs is a *well-formedness condition*: it makes no sense for *function definitions* to omit their names, and doing so should cause the compiler to complain (as JavaScript engines do). But there is no particular reason to make this a *syntax* error. One can parse all function-looking code into a `Function` AST node with an optional name, and then separately validate that such nodes, when in statement position, have a name. This yields both simpler parsers and better error messages.

A similar situation arises with lambda expressions, which in JavaScript are written $(x_1, x_2, \dots, x_n) \Rightarrow e$. The issue is that $(x_1, x_2, \dots, x_n)$ on its own is also a valid expression, using the “comma operator” (a legacy syntax from C to sequence side effects). In our perspective, JavaScript has *comma-separated lists of expressions*; depending on context, these lists can be either lambda parameter lists or repeated applications of the comma operator. It just so happens that the expressions in a lambda parameter list are restricted (i.e., destructuring patterns) – we think this is best understood as a *semantic* distinction, rather than a syntactic one¹¹. This distinction matters in practice. For example, upon writing the ill-formed lambda expression $(x, 1) \Rightarrow x$, the programmer is rewarded by the Node.js JavaScript engine with the cryptic

```
Uncaught SyntaxError: Invalid destructuring assignment target
```

despite no assignments being involved. Had one used a simpler, lexical approach, the error message could have been more informative by construction.

The Pratt philosophy outlined above has not been as influential as one could have hoped. As Crockford wrote [6] in 2007:

Oddly enough, such an obviously utopian approach to compiler construction is completely neglected today. (Douglas Crockford)

Crockford went on to contribute a beautiful JavaScript implementation of Pratt’s proposed design, albeit sticking to a simplified subset of JavaScript – and thereby avoiding the ambiguities mentioned above – because, in the author’s own words, “the language is a mess”.

4.2 A New Perspective

Pratt’s design was originally implemented as a proto-object-oriented technique, attaching code directly to token instances using the dynamic features of its implementation language Lisp. This had the advantage of making the parser easily extensible. While this approach was faithfully reproduced by Crockford [6], more recent accounts of similar techniques, for example by Hanson [12], Kladov [18], and Rompf [24] have used a more straightforward mix of recursion and imperative loops, usually forgoing extensibility.

Naturally, our approach has important limitations. At its core, it is little more than a glorified LL(1) parser with a few twists. One twist is that it uses *precedence* (a.k.a. *binding power*) pervasively, and not just for operators, which allows resolving many of what would otherwise be LL(1) ambiguities. Another twist is that we rely on the “maximal munch” principle that already exists in the domain of lexical analysis: this means that the parser will always try to parse as much as possible eagerly, which resolves the “follow-set” conflicts of LL(1) approaches

¹¹The ECMAScript(R) 2026 Language Specification thinks otherwise: it defines a production called *CoverParenthesizedExpressionAndArrowParameterList*. It makes its syntax definition context-sensitive via “cover grammars”, whose interpretation is later *refined* [14].

by just picking a sane default. This means, for instance, that `if a1 then if a2 then b else c` will always parse equivalently to `if a1 then (if a2 then b else c)`, which is the most common interpretation in real programming languages. In fact, we are not aware of any situation where the maximal munch principle applied to parsing “does the wrong thing”. A final twist is that we support some self-referential rules, which are a form of left recursion (see Section 3.3); together with the reliance on precedence, this allows defining expressions naturally, an observation also made by, e.g., Parr [22].

A key takeaway of our paper is that, for a broad class of *mainstream* surface syntaxes – namely, syntaxes that are tokenizable and whose disambiguation can be decided with bounded lookahead and without semantic information – it should be possible to define most constructs using Pratt’s “lexical semantics” philosophy. Then, by following our approach to designing the parser, much of what users of the language have to know about its syntax can be derived automatically, namely the rule diagrams and the precedence table.

4.3 Implementation Discussions

A nice benefit of defining most parsing logic through uniform rules is that we can manipulate these rules programmatically, for pretty printing, debugging, error reporting, etc.

Tracing and Debugging

Besides inspecting constructed rules directly, we can trace the parser at the few points where the rule interpreter makes a decision. The first useful event is simply the next token consumed by the parser.

```
fun consume = if tokens is head :: tail then
  tracer.print of "consume: ~" + Token.summary(head) + "~"
  set tokens = tail
```

Rule parsing is wrapped in a trace that reports both the rule being entered and the tree produced by that rule.

```
fun parseRule(prec: Int, rule) = tracer.trace of
  "parsing rule " + rule.name + " with precedence " + prec
  result => "parsed rule " + rule.name + ": " + result Tree.summary()
  () => if tokens is
  // ...
```

Inside that function, keyword choices are the next useful branch to log: the trace tells us which token was recognized as a keyword and which rule branch it selected.

```
token :: _ and Token.name(token) is Some(name) and
do tracer.print of "found token " + name
  rule.keywordChoices |> MutMap.get(name) is Some(rest) then
  tracer.print of "found keyword branch " + name
  consume
  parseRule(0, rest)
  // ...
  // ...
```

The continuation trace focuses on association decisions. It prints the accumulated tree, then reports the candidate operator and its two binding powers before deciding whether the continuation is allowed.

```

fun parseCont(acc: Tree, prec: Int, kindName: Str, rule, cont) = if tokens is
do tracer.print of ">>> " + kindName + "Cont " + prec + " " + acc Tree.summary()
Token.Symbol(name) :: _ and opInfo(name) is Some([leftPrec, rightPrec]) then
do tracer.print of "found operator " + name
do tracer.print of "leftPrec = " + leftPrec + "; rightPrec = " + rightPrec
// ...
// ...

```

The resulting traces expose source-level parsing decisions: which token was consumed, which rule was active, which branch matched, and which binding powers allowed or blocked a continuation. This is the information one needs to debug a failed parse or a wrong association. The Ultimate Conditional Syntax [4] helps because `do` directives can appear inside conditional expressions, so this instrumentation remains ordinary source code rather than machinery generated by a parser tool.

Avoiding Mutation

Mutation is used in this paper only for conciseness: a pure parser can thread the token stream through recursive calls, manually or via a monad or algebraic effects; using `LazyStack` instead of `Stack` keeps it streaming. For example, `expr` can return the parsed tree together with the unconsumed token stream.

```

fun expr(tokens, bp): [Tree, Stack[Token]] = if tokens is
Token.Literal(kind, literal) :: rest then [Tree.Literal(kind, literal), rest]
// ... many cases omitted for brevity
Nil then [Tree.Error(None, "Unexpected end of input"), tokens]

```

5 Related Work

There has been a more recent push towards simplicity in the design of parsers and parsing tools. On one hand, language-integrated *parser combinators* [13] reduce the friction of defining parsers and typically side-step formal grammar specifications by using mechanisms based on *packrat parsing* [9]. The first matching parser rule is chosen, and if it succeeds, the parser advances to the next rule without backtracking. These approaches replace the problem of grammatical *ambiguity* with the problem of specifying the parsing clauses in an order that allows for the desired programs to be parsed, a less principled approach that requires more testing and debugging.

Numerous approaches have been proposed to address the challenges of parsing complex language features. For example, [7] introduce a modular grammar scheme for mixfix operators based on a directed acyclic graph of precedences, enabling more flexible parser construction. [10] proposed Parsing Expression Grammars (PEGs), which offer a powerful alternative to context-free grammars (CFGs) by relying on recognition-based rather than generative syntax definitions, thereby eliminating ambiguity at its core. [19] extend PEGs to handle left recursion, associativity, and precedence rules, while [2] develop a verified packrat parser interpreter that leverages the clarity of PEGs for formal correctness. Looking more generally at parser combinators, [17] present general parser combinators that support arbitrary context-free grammars, including ambiguous ones.

Another line of work focuses on robustly enabling extensible syntax. Techniques like *enforestation*, as seen in projects like Rhombus [8], which aim for highly extensible language syntaxes, also take a similar approach to top-down operator precedence parsing.

6 Conclusion

In this paper, we proposed a simple recipe for constructing parsers to parse simple computer language syntaxes. We showed how to extend a basic Pratt parsing implementation to user-defined parsing rules in a way that could even be made extensible by users of the language itself, since the parsing categories and rules can be dynamically extended. We used a variation on Caml Light as an example of a syntax that is relatively straightforward to parse using our approach. Nevertheless, we found a few cases of Caml Light syntax features that require some ad-hoc handling by direct modification of the parsing code, such as its handling of top-level let bindings and expressions, which are ambiguous. The fact that we could still support this ambiguous syntax through some manual intervention is a testament to the flexibility of our approach. Unsurprisingly, these features are also ones that tend to confuse users; thus, we consider them to be design flaws, leading to our postulate that a syntax that is not easy to parse in our approach is not a very good syntax in the first place.

Our approach to parsing was born out of a desire to make the MLscript parsing algorithm as simple and easy to develop and maintain as possible. We are currently porting MLscript's parser to this approach.

Indeed, the first versions of the MLscript parser used a parser combinator library, which was found to be very hard to work with, as it was difficult to verify what grammar the parsing rules were implementing and to debug them when programs did not parse as expected. The next version of MLscript used a handwritten recursive descent parser following the “*just write the #!%/ * parser*” philosophy [24], but it was full of ad-hoc parsing rules and special cases. Although it was very easy to write and was much easier to debug, the MLscript designers ended up facing the same challenges discussed above. Finally, a system of parsing rules very similar to the one described in this paper ended up being designed. That system is still in flux, and a lot of ad-hoc logic could be moved from recursive definition into proper parsing rules, as we did in this paper for Caml Light. A major difference between Caml Light and MLscript is that the latter is deeply indentation-sensitive; however, MLscript addresses this by generating INDENT and DEINDENT tokens and preprocessing the token stream.

References

- 1 Tab Atkins Jr. railroad-diagrams: A small js+svg library for drawing railroad syntax diagrams, 2025. GitHub repository, commit 8b92af01cca06afb65895ae29964d84a952fed08. Accessed 2026-02-07. URL: <https://github.com/tabatkins/railroad-diagrams>.
- 2 Clement Blaudeau and Natarajan Shankar. A verified packrat parser interpreter for parsing expression grammars. In Jasmin Blanchette and Catalin Hritcu, editors, *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2020, New Orleans, LA, USA, January 20-21, 2020*, CPP 2020, pages 3–17, New York, NY, USA, 2020. ACM. doi:10.1145/3372885.3373836.
- 3 Luyu Cheng and Lionel Parreaux. Online Web Demonstration of A Simple Recipe for Writing Decent Recursive Descent Parsers. InteractiveResource (visited on 2026-06-16). URL: <https://recursive-descent-parsing.mlscript.dev>, doi:10.4230/artifacts.26720.
- 4 Luyu Cheng and Lionel Parreaux. The ultimate conditional syntax. *Proc. ACM Program. Lang.*, 8(OOPSLA2), October 2024. doi:10.1145/3689746.
- 5 Shardul Chiplunkar and Clément Pit-Claudel. Automatic layout of railroad diagrams, 2025. doi:10.48550/arXiv.2509.15834.
- 6 Douglas Crockford. *Top Down Operator Precedence*, chapter 9, pages 129–146. O'Reilly Media, 2007.

- 7 Nils Anders Danielsson and Ulf Norell. Parsing mixfix operators. In Sven-Bodo Scholz and Olaf Chitil, editors, *Implementation and Application of Functional Languages - 20th International Symposium, IFL 2008, Hatfield, UK, September 10-12, 2008. Revised Selected Papers*, volume 5836 of *Lecture Notes in Computer Science*, pages 80–99, Berlin, Heidelberg, 2008. Springer. doi:10.1007/978-3-642-24452-0_5.
- 8 Matthew Flatt, Taylor Allred, Nia Angle, Stephen De Gabrielle, Robert Bruce Findler, Jack Firth, Kiran Gopinathan, Ben Greenman, Siddhartha Kasivajhula, Alex Knauth, Jay McCarthy, Sam Phillips, Sorawee Porncharoenwase, Jens Axel Sogaard, and Sam Tobin-Hochstadt. Rhombus: A new spin on macros without all the parentheses. *Proc. ACM Program. Lang.*, 7(OOPSLA2), October 2023. doi:10.1145/3622818.
- 9 Bryan Ford. Packrat parsing: simple, powerful, lazy, linear time, functional pearl. *SIGPLAN Not.*, 37(9):36–47, September 2002. doi:10.1145/583852.581483.
- 10 Bryan Ford. Parsing expression grammars: a recognition-based syntactic foundation. *SIGPLAN Not.*, 39(1):111–122, January 2004. doi:10.1145/982962.964011.
- 11 Dick Grune and Ceriel J. H. Jacobs. *Parsing Techniques - A Practical Guide*. Monographs in Computer Science. Springer, New York, NY, 2 edition, 2008. doi:10.1007/978-0-387-68954-8.
- 12 David R Hanson. Compact recursive-descent parsing of expressions. *Software: Practice and Experience*, 15(12):1205–1212, 1985. doi:10.1002/spe.4380151206.
- 13 Graham Hutton and Erik Meijer. Monadic parsing in haskell. *J. Funct. Program.*, 8(4):437–444, 1998. doi:10.1017/s0956796898003050.
- 14 Marja Hölttä. Understanding the ecma script spec, part 4, 2020. V8 blog post. Accessed 2025-06-03. URL: <https://v8.dev/blog/understanding-ecmascript-part-4>.
- 15 INRIA. Caml light. <https://caml.inria.fr/caml-light/>. Last updated 2004-05-01. Accessed 2026-02-07.
- 16 INRIA. The core caml light language. <https://caml.inria.fr/pub/docs/manual-caml-light/node3.html>, 1997. Caml Light reference manual, core language chapter.
- 17 Anastasia Izmaylova, Ali Afrozeh, and Tijs van der Storm. Practical, general parser combinators. In Martin Erwig and Tiark Rompf, editors, *Proceedings of the 2016 ACM SIGPLAN Workshop on Partial Evaluation and Program Manipulation, PEPM 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, PEPM '16, pages 1–12, New York, NY, USA, 2016. ACM. doi:10.1145/2847538.2847539.
- 18 Alex Kladov. Simple but powerful pratt parsing, 2020. Accessed 2025-02-28. URL: <https://matklad.github.io/2020/04/13/simple-but-powerful-pratt-parsing.html>.
- 19 Nicolas Laurent and Kim Mens. Parsing expression grammars made practical. In Richard F. Paige, Davide Di Ruscio, and Markus Völter, editors, *Proceedings of the 2015 ACM SIGPLAN International Conference on Software Language Engineering, SLE 2015, Pittsburgh, PA, USA, October 25-27, 2015*, SLE 2015, pages 167–172, New York, NY, USA, 2015. ACM. doi:10.1145/2814251.2814265.
- 20 OCaml. 7 expressions, 2023. The OCaml Language manual, version 5.1. Accessed 2026-02-04. URL: <https://ocaml.org/manual/5.1/expr.html>.
- 21 Francisco Ortin, José Quiroga, Oscar Rodríguez-Prieto, and Miguel García. An empirical evaluation of lex/yacc and antlr parser generation tools. *PLoS ONE*, 17, 2022. doi:10.1371/journal.pone.0264326.
- 22 Terence Parr and Kathleen Fisher. Ll(*): the foundation of the ANTLR parser generator. In Mary W. Hall and David A. Padua, editors, *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2011, San Jose, CA, USA, June 4-8, 2011*, PLDI '11, pages 425–436, New York, NY, USA, 2011. ACM. doi:10.1145/1993498.1993548.
- 23 Vaughan R. Pratt. Top down operator precedence. In Patrick C. Fischer and Jeffrey D. Ullman, editors, *Conference Record of the ACM Symposium on Principles of Programming Languages, Boston, Massachusetts, USA, October 1973*, POPL '73, pages 41–51, New York, NY, USA, 1973. ACM Press. doi:10.1145/512927.512931.

- 24 Tiark Rompf. Just write the `#!%/*` parser, 2022. Accessed 2024-04-10. URL: <https://tiarkrompf.github.io/notes/?/just-write-the-parser/>.
- 25 Scala Language. Expressions. Scala 2.13 Language Specification. Accessed 2026-02-04. URL: <https://scala-lang.org/files/archive/spec/2.13/06-expressions.html>.
- 26 Boštjan Slivnik and Marjan Mernik. On parsing programming languages with turing-complete parser. *Mathematics*, 11(7), 2023. doi:10.3390/math11071594.
- 27 Joe Zimmerman. Practical LR parser generation. *CoRR*, abs/2209.08383, 2022. doi:10.48550/arXiv.2209.08383.